SIMATS ENGINEERING COLLEGE

Department of Information Technology

ASSESSMENT TOOL 3 — REVERSE ENGINEERING TASK

Course: Object Oriented Analysis and Design | Code: CSA1142 | CO4 | Marks: 30

Reg. No.: 192511205	Name: Krithika	Q.No.: Q4	S.No: 6/37	Date: 24-06-2026

Question 4: Data Access Layer with DAO Pattern

DAO Pattern | OODBMS | Decoupling | Access Layer

Bloom's Level: BL4 – Analyse & Apply

Scenario:

An application directly accesses the database from multiple classes causing redundancy and tight coupling. SQL queries are scattered across business classes, making any change to the database schema require updates in dozens of locations. Object persistence is inconsistent and error-prone, leading to data integrity issues.

Question (Marks: 30):

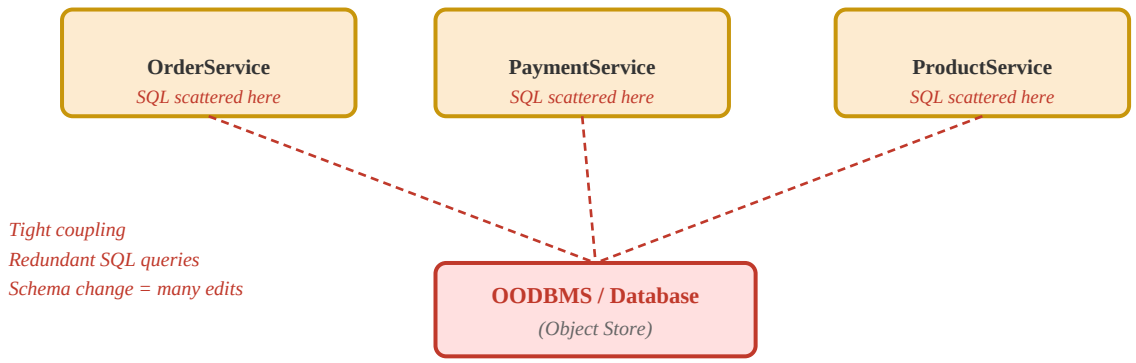
Reverse engineer the system by introducing a Data Access Layer (DAO pattern) and explain how it improves integration with OODBMS.

Answer:

1. Introduction

The given scenario describes a tightly coupled, anti-pattern system where business logic classes directly contain SQL/OQL queries and communicate with the database without any abstraction layer. This violates the Single Responsibility Principle (SRP) and the Dependency Inversion Principle (DIP), resulting in poor maintainability, testability, and scalability. The solution is to apply the Data Access Object (DAO) Pattern — a structural pattern that encapsulates all data access logic inside dedicated classes, exposing a clean interface to the rest of the application.

Figure 1: BEFORE — Tightly Coupled System (No DAO, SQL scattered across classes)



2. Problems in the Existing System (Before Reverse Engineering)

• Tight Coupling: Every service class (OrderService, PaymentService, ProductService) directly holds database query strings, making them impossible to test independently.

• Redundancy: The same OQL/SQL query (e.g., "SELECT p FROM Product p WHERE p.id = ?") is duplicated across multiple classes, violating the DRY (Do Not Repeat Yourself) principle.

• Brittleness: A single schema change (e.g., renaming a field in the Product class) forces edits in every class that references it — potentially dozens of locations.

• Inconsistent Persistence: Without a centralised persistence layer, object state may be saved partially or in an incorrect order, causing data integrity violations in the OODBMS.

• No OODBMS Benefits: Using raw SQL-style strings instead of OQL negates the polymorphism and object-graph traversal advantages of an Object-Oriented Database Management System.

3. Reverse Engineering Solution — Introducing the DAO Pattern

Reverse engineering the system means restructuring existing code to introduce a Data Access Layer (DAL) without changing the external behaviour of the application. The transformation steps are:

• Step 1 — Define a Generic DAO Interface: Create a generic interface GenericDAO<T> declaring CRUD operations: findById(), findAll(), save(), delete(). All business logic depends only on this abstraction.

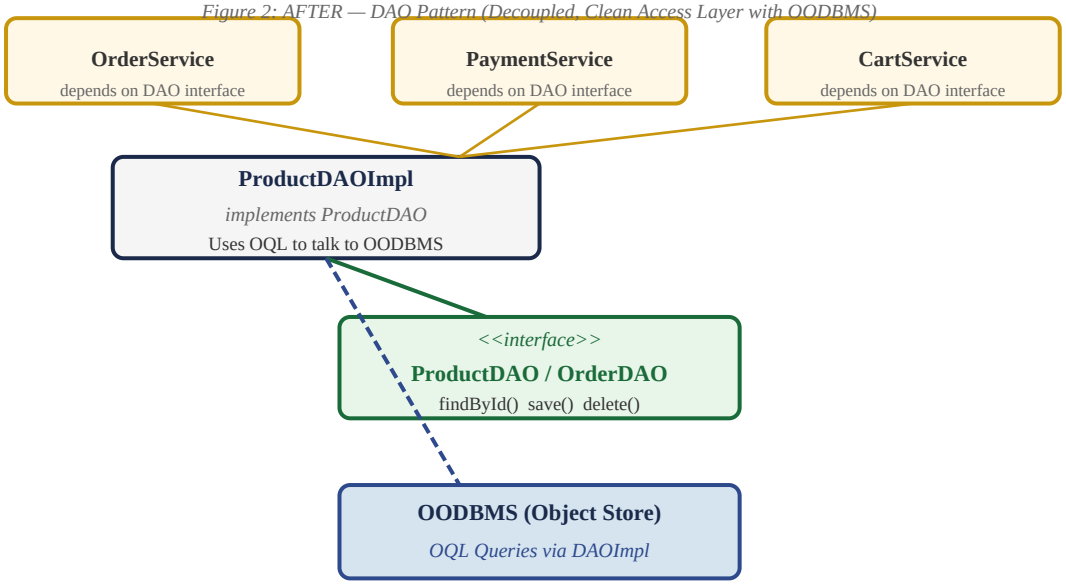
• Step 2 — Create Entity-Specific DAO Interfaces: Extend the generic interface for each domain entity: ProductDAO, OrderDAO, CustomerDAO. Add entity-specific query methods such as findByCategory() or findByCustomerId().

• Step 3 — Implement DAOs using OODBMS OQL: Write ProductDAOImpl, OrderDAOImpl that use Object Query Language (OQL) to communicate with the OODBMS, returning fully hydrated domain objects — not flat rows.

• Step 4 — Refactor Business Classes: Remove all direct database calls from OrderService, PaymentService, etc. Inject the DAO interface via constructor injection (Dependency Inversion).

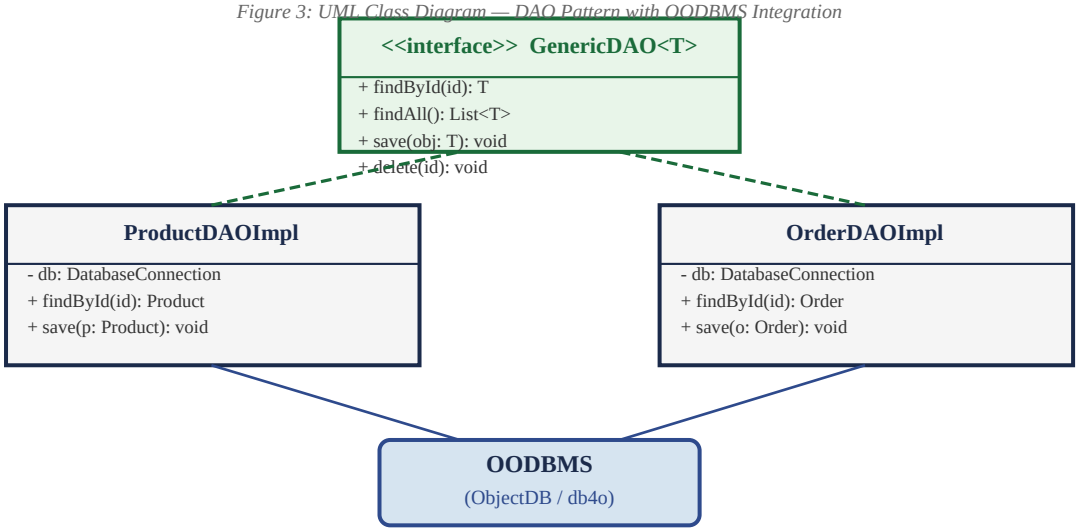
• Step 5 — Centralise Connection: Wrap the OODBMS connection inside a Singleton DatabaseConnection class used by all DAOImpl classes.

Figure 2: AFTER — DAO Pattern (Decoupled, Clean Access Layer with OODBMS)



4. UML Class Diagram — DAO Pattern Components

Figure 3: UML Class Diagram — DAO Pattern with OODBMS Integration



The GenericDAO<T> interface provides a type-safe contract. Both ProductDAOImpl and OrderDAOImpl implement this interface and use the shared OODBMS connection to execute OQL queries. The dashed arrows denote implementation relationships; solid arrows

denote dependency (uses) relationships.

5. Sequence Diagram — DAO Interaction with OODBMS

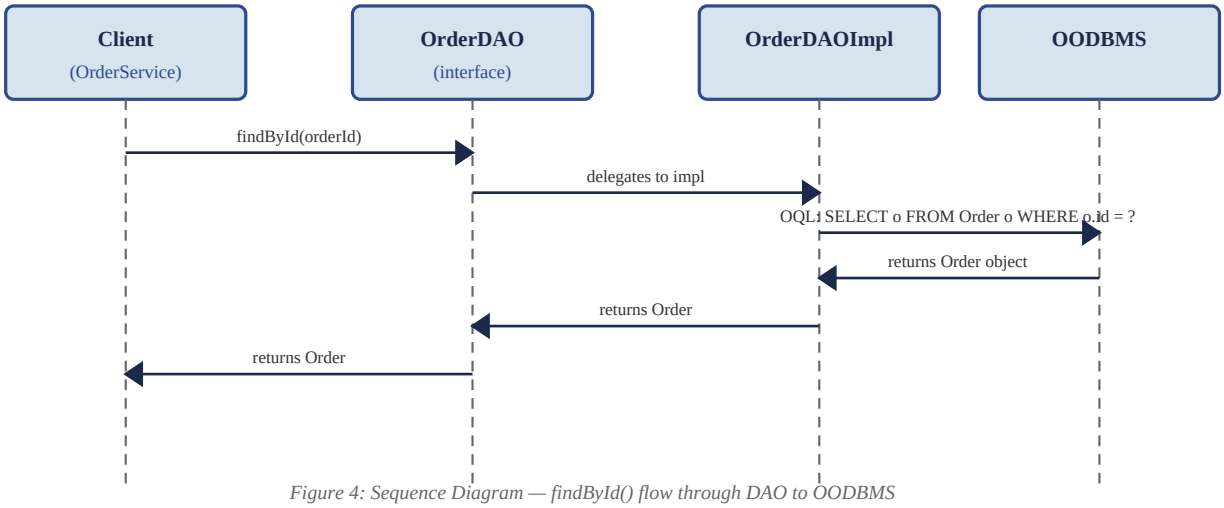


Figure 4: Sequence Diagram — findById() flow through DAO to OODBMS

When OrderService calls orderDAO.findById(orderId), the call is routed through the DAO interface to OrderDAOImpl, which constructs an OQL query and submits it to the OODBMS. The OODBMS returns a complete Order object graph (with nested Products and Customer), eliminating the need for JOIN operations or manual object assembly.

6. DAO Implementation — Code Illustration

GenericDAO Interface:

public interface GenericDAO<T> { T findById(Object id); List<T> findAll(); void save(T obj); void delete(Object id); }

ProductDAOImpl using OODBMS (ObjectDB / OQL):

public class ProductDAOImpl implements ProductDAO { private final EntityManager em = DatabaseConnection.getInstance().getEM(); @Override public Product findById(Object id) { return em.find(Product.class, id); // OODBMS returns full object graph } @Override public void save(Product p) { em.getTransaction().begin(); em.persist(p); em.getTransaction().commit(); } }

7. How DAO Pattern Improves OODBMS Integration

- Native Object Persistence: DAOImpl uses JPA/OQL API directly, storing and retrieving Java objects without any O-R mapping overhead. The OODBMS preserves inheritance hierarchies and associations natively.
- Object Graph Traversal: A single orderDAO.findById() call returns the full Order object including nested Products and Customer — no JOIN queries needed.
- Polymorphic Queries: OQL supports polymorphism: querying Product automatically includes DiscountedProduct subclass objects stored in the same OODBMS extent.
- Transaction Safety: DAOImpl methods wrap operations in OODBMS transactions, ensuring ACID compliance and preventing the partial-save data integrity problems seen in the original system.
- Schema Evolution: If the Product class gains a new field, only ProductDAOImpl needs updating — all calling business classes remain unchanged.

8. Before vs. After — Comparison Matrix

Aspect	Before (No DAO)	After (DAO Pattern)
Coupling	Tight — SQL in business classes	Loose — via DAO interface
Maintainability	Schema change = dozens of edits	Change only DAOImpl
Testability	Cannot mock DB easily	Mock DAO interface in tests
Reusability	Duplicated SQL logic	Centralised DAO methods
OODBMS Support	Raw SQL (wrong for OODBMS)	OQL via DAOImpl, objects returned

Scalability	Hard to swap DB technology	Swap impl class only

9. Conclusion

By reverse engineering the tightly coupled application and introducing the DAO pattern, the system achieves a clean separation between business logic and data persistence. The DAO interface provides a stable contract that allows the underlying OODBMS implementation to change without affecting any service class. Integration with the OODBMS is significantly improved: OQL queries return complete object graphs, transactions are centralised inside DAOImpl, and polymorphism is preserved natively. The result is a maintainable, testable, and scalable data access architecture that fully exploits the strengths of an Object-Oriented Database Management System.

Evaluation Rubric (Summary)

Criteria	Weightage	L4 – Exemplary	L1 – Beginning	Marks
Understanding of System	20%	Clear, in-depth understanding	Lacks understanding	/6
Decomposition	25%	Effective, detailed analysis	Unable to decompose	/7.5
Identification of Components	20%	Accurately identifies all	Cannot identify	/6
Reconstruction	20%	Successfully reconstructs	Unable to reconstruct	/6
Documentation	15%	Well-organized, with diagrams	No proper documentation	/4.5

Declaration: I certify that this submission is my original work and I have adhered to the guidelines specified for this assessment.

Signature: Krithika Staff Initial: _____ **Total Marks: _____ / 30**